

Good — but I'm going to tighten this into something real and buildable, not just theory. What you're describing is basically a “**MLB Prop Edge Engine**” MVP → model → product.

I'll give you the full stack in 3 layers:

LAYER 1 — MVP APP (what you build first)

Goal

Start with ONE market:

Pitcher strikeouts (Over/Under)

Not parlays. Not HRs. Not everything.

MVP FEATURES

Input

- Pitcher name
- Opposing team
- Game date
- Strikeout line (e.g. 6.5)

Output

- Expected Ks
 - Probability over/under
 - Fair odds
 - Sportsbook odds (manual input at first)
 - Edge %
-

SIMPLE UI (web app)

- Left: select pitcher + game
- Middle: model output
- Right: “value bets detected”

Example:

Dylan Cease vs NYY

Line: 7.5 Ks

Model:

Expected Ks: 6.9

Over probability: 41%

Fair odds: +144

Sportsbook:

-120

EDGE: ❌ No bet

TECH STACK (simple)

- Frontend: React or Next.js
 - Backend: Python (FastAPI)
 - Database: SQLite (start) → Postgres later
 - Data: MLB Stats API + FanGraphs
-

LAYER 2 — YOUR STRIKEOUT MODEL (CORE EDGE)

This is where your real value comes from.

STEP 1 — Expected Strikeouts (λ)

You calculate:

$$\lambda = (K/9) \times \text{IP} \times \text{Matchup Factor}$$

Where:

- $K/9$ = pitcher strikeout ability
 - IP = expected innings pitched (VERY important)
 - Matchup Factor = opponent strikeout tendency
-



STEP 2 — Probability Model

Use Poisson:

$$P(X=k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Then compute:

- $P(\text{Over } 6.5 \text{ Ks}) = P(7+)$
 - $P(\text{Under } 6.5 \text{ Ks}) = P(0-6)$
-



STEP 3 — Convert to Fair Odds

If:

- $P(\text{over}) = 0.58$

Then:

- Fair odds = $1 / 0.58 = 1.72$ decimal (~ -139)

Compare vs sportsbook.



STEP 4 — EDGE SCORE

$$\text{Edge} = \text{Fair Odds} - \text{Sportsbook Odds (implied)}$$

Only bet if:

- Edge > 3–5%

LAYER 3 — DATA PIPELINE (what makes it powerful)

DATA YOU NEED

Pitcher data

- K%
- K/9
- SwStr% (very important)
- Avg innings per start
- Pitch count per game

Opponent data

- Team K% vs handedness
- OBP of top 1–5 hitters
- Strikeout vs pitch types

Context

- Ballpark factor
- Weather (temp, wind)
- Vegas implied runs (important hidden signal)

SOURCES

- MLB Stats API (free)
 - FanGraphs (advanced stats)
 - Baseball Savant (pitch data)
 - Odds API (sportsbook lines)
-

LAYER 4 — “SHARP VERSION” (what pros eventually add)

Once MVP works:

1. Pitch count projection model

Instead of guessing innings:

- model pitch efficiency
- strikeout pitch usage
- manager tendencies

2. Live betting module

- after 2 innings:
 - velocity drop
 - swing-and-miss rate
 - fatigue signals

3. Line movement tracker

- detect when books move early
- flag “sharp money”

4. Market comparison engine

Compare:

- DraftKings
- FanDuel
- bet365

Find best line automatically.

WHAT ACTUALLY MAKES YOU MONEY (IMPORTANT)

Not:

- predicting winners

But:

detecting **small mispriced pitcher strikeout lines before they move**

Examples:

- Book sets 7.5 Ks too high for a low pitch-count pitcher
 - Weather boosts strikeout environment but line doesn't adjust
 - Opponent lineup changes last minute
-

REALITY CHECK

Even good models:

- win 52–56% long-term
- profit comes from small edges + volume discipline

This is not lottery betting — it's:

pricing inefficiency detection

IF YOU WANT NEXT STEP

I can build you:

1. Full Python MVP code (ready to run)

- data pull
- strikeout model
- edge calculator

2. Database schema + API design

3. React UI mock + layout

4. Or “version 2” AI model (better than Poisson)

- regression model (XGBoost)
- pitch-level simulation
- Monte Carlo outcomes

Just tell me:

👉 “build MVP code”

👉 or “make it production app”

👉 or “make it sharp betting system”

and I’ll take it to the next level.